

9 - Evaluation

Contents

9.1 Effectiveness of the Programming Language	2
9.2 Compare & Contrast with Similar Commercial Systems	4
Google Forms	4
Digital Unite	5
Official DVSA Theory Test Kit App	6
9.3 Identify Good Features and Shortcomings	7
9.4 Significant Potential Improvements	9
9.5 Own Strengths & Weaknesses during Design & Development	10
9.6 Future Changes of Approach to Avoid Problems	12

9.1 Effectiveness of the Programming Language

Java was the programming language that I chose to complete my project. I first learned Java as part of the A Level computer science course. Java has many features that were well-suited to my project.

The first of which is the fact that it's a compiled language. This means that the code executes more quickly than an interpreted language which was important because the system would be running on slow laptops and so any speed improvement was an advantage. It also meant that any external libraries that I used could be packaged with the executable. I ended up using three external libraries and so, as they are packaged with the executable, the system is able to run on computers which don't have those libraries installed.

Even though Java is compiled, the byte code is run by the Java Virtual Machine (JVM) rather than directly on the system's CPU. This means that my program was very portable and could run on many different operating systems. This was useful because it meant that I could develop on my Linux PC at home and there were no issues when running on Windows computers. It was also necessary that the program was cross-platform because some of the laptops in the club run Windows 7 and some run Windows 10. This meant that I didn't need to produce separate versions of the program for different operating systems and sped up development.

Java is an object-oriented programming language. This had a few advantages. Firstly, it made handling data objects simpler as each data entity could have its own object and then all data related to that entity could easily be passed around by passing around the object. The data could be treated as a single unit.

Inheritance was also a very useful feature. It meant that I could extend and create subclasses of the standard classes by adding new methods. This meant that I could add validation functionality to the built in GUI components, e.g. text fields, without having to write a text field from scratch myself.

Encapsulation was also a very useful feature because it meant that I could use the objects in different parts of my program without having to worry about how they were carrying out complex tasks behind the scenes, e.g. being able to call `questionStats.getByID()` without having to worry about the lookup in the Hash Table that was taking place behind the scenes. This reduced the complexity of my program and made it easier to read.

It also meant that I could reuse objects. This sped up development time considerably. This is because I could create easily reusable GUI components. For example, I created a question selection component and reused it in the create forms, statistics, and import and export panels. This also meant that if I wanted to change the component slightly, e.g. change the size of the text, I only had to change it once and the changes were reflected across all of the panels.

Java is a high level programming language and so it's very close to English. This meant that it was easier to program in than a lower level language. It also meant that I could use built-in commands that perform complex tasks e.g. `Math.Random` to generate random numbers that I used to create unique IDs for the questions and the forms.

Java is also strongly typed. This means that you have to explicitly declare the data type of each variable and only data of that type can be stored in that variable. This made handling data easier because it meant that I knew exactly what sort of data was stored in each variable. However, it also slowed down development as it was tedious and time-consuming to have to declare the type of each variable before using it.

I used the Swing and AWT libraries for creating my GUI. These provided a large amount of built-in components that could be used. The most useful of which were the wide variety of entry components, e.g. text fields and comboboxes. This meant that I could offer a wide variety of entry types that the user could use to create questions and forms, without having to create any from scratch myself. The wide variety of layout managers provided meant that I could make my system fully resizable and arrange all components exactly how I wanted. However, although the large amount of layout managers was useful, I would have preferred a small number of layout managers that were more versatile/powerful as this would have been less complicated and easier to program. I also found developing the GUI without using an IDE very tedious and time-consuming because of the large number of constraints that had to be specified for each object. However, I managed to become quite fast at it in the end.

The built-in tables that Swing provided were very useful as it made it very easy to add tables to my system. However, making the rows dynamically resize so that the text in them was always visible was very difficult and there was no built-in solution. This took a large amount of time for me to develop and I think that it would be better if it were a built-in feature because automatically resizing tables is a feature that most implementations of tables require.

Java.IO was another useful library that I used. It made interacting with files very simple. This was essential for my system because it meant that all data could be saved between system restarts. It also made implementing the import and export questions and forms features easier. This is because all of the code for low level manipulation of files was prewritten.

There were also a large number of third party libraries that were available for me to use. This is because Java is very widely used.

I used the LGoodDatePicker library to easily add a date picker component. Swing doesn't have a calendar entry component built in so using this library saved me a large amount of time because I didn't have to develop the datepicker myself. It was already well-tested so there were no/few bugs.

JFreeChart was a very useful library also. This allowed me to quickly and easily add graphs and charts to my program. This was used for the view statistics screen.

Finally, I used the google maps API library in order to use the PlacesAutocompleteAPI which allowed me to implement the location entry that auto completes the address that the user is entering.

There was excellent documentation for all libraries (built-in and third party) that I used. This is because there is a standardised documentation format for Java called Javadoc. This documentation meant that I could find out how each library should be used and implement them correctly. As the format is standardised, it was easy to navigate through all of the documentation because they were all laid out in the same way.

I made extensive use of the site "stackoverflow.com". There were answers to almost all questions that I had about Java on it. This aided me in overcoming various problems that I encountered during development. There was such an extensive database of answers because of how widely-used Java is.

I developed my system using the open source text editor Notepad++. This was a fairly basic editor however it had a few very useful features. The best of which was syntax highlighting; this made the code much easier to read and helped me spot errors. Notepad++ also allows you to edit multiple lines at once which made repetitive actions such as adding action listeners to buttons faster and so saved time. I also used this in conjunction with the find and replace function.

9.2 Compare & Contrast with Similar Commercial Systems

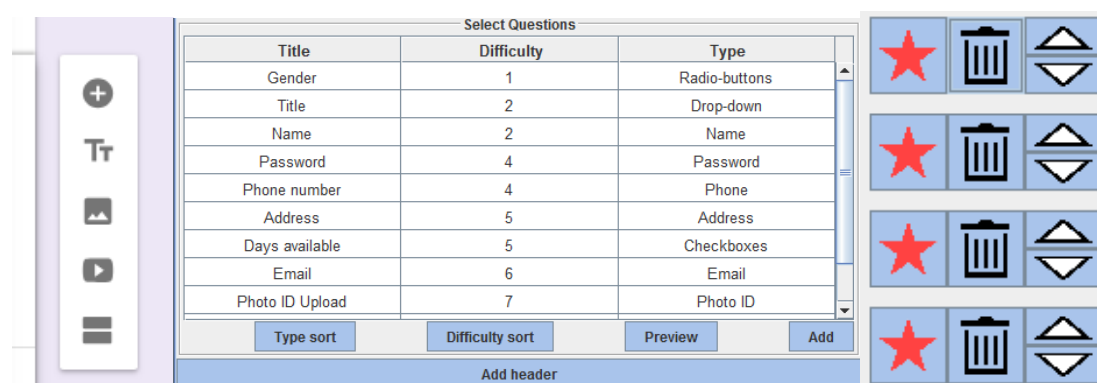
There are currently no commercial system that offers the same functionality as my system. However there are a few systems that offer elements of what my system does.

Google Forms



I investigated the form-builder aspect of Google Forms during my investigation.

I didn't like the five add component buttons at the side because it wasn't clear what they did. I improved upon this by making the purpose of all the buttons in my system clear.



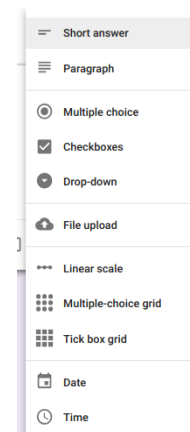
Google forms is on the left and my system is on the right. The only button of mine that I think could be seen as unclear is the red star. However, the bin icon to delete questions and the arrows to move questions up and down the form have very clear meaning. The select questions table is also easy to use because all buttons are clearly labelled.

I also disliked that, when adding a question to a form in Google Forms, you always had to add a multiple choice question and then change the type to a different question because I thought it was confusing. In my system you can just immediately add the type of question you want to the form.

I liked the icons that Google Forms provided for each question type as they made it very easy to see what each question was. These weren't implemented in my system however I think that the title and type of question that my system displays is equally useful.

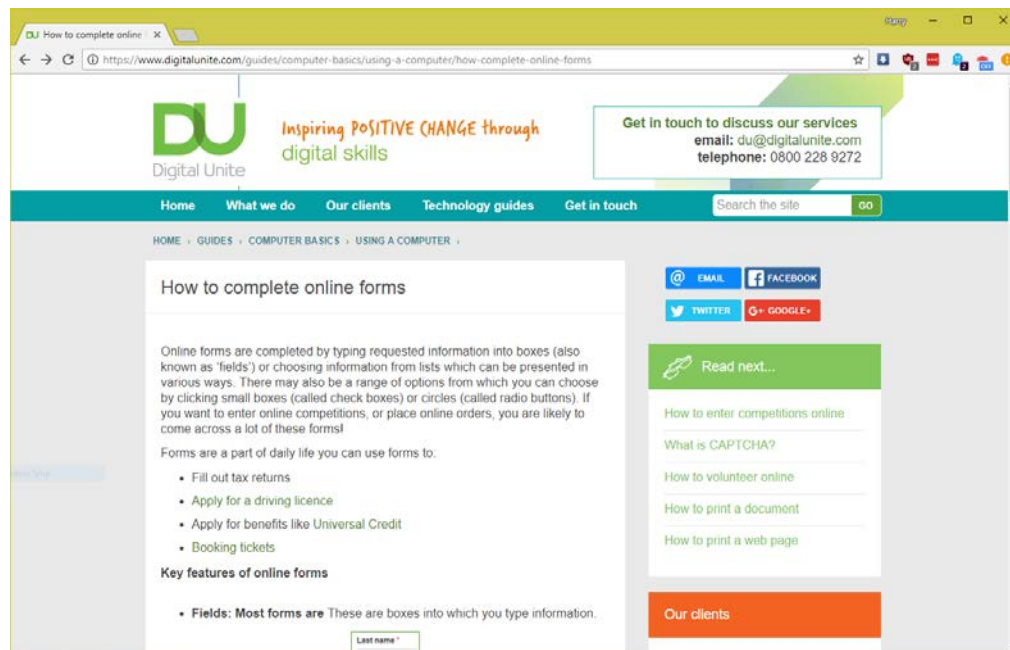
In Google Forms, you have to create all of the questions, which you want in the form, inside each form. There is no facility to create questions once and reuse them in other forms. There are no premade questions therefore creating multiple forms at once can be quite slow. My system improves on this because the user only has to make each question once and can use it in as many forms as they want.

I liked the method of adding options to all of the types of questions that require it (radio buttons, checkboxes, dropdown) as it was consistent between all of the different entry types. In my system, I made the option entry for each entry that required it the same to match this good feature of Google Forms.



To rearrange the questions in Google Forms, you can drag and drop them. This is good for when you're moving a question for a short distance but can be difficult when you have to move the question a longer distance because the window scrolls, making it hard to accurately place the question. My system has an easier to use method of moving questions because the user can move the questions by pressing an up or down button without fiddly dragging.

Digital Unite



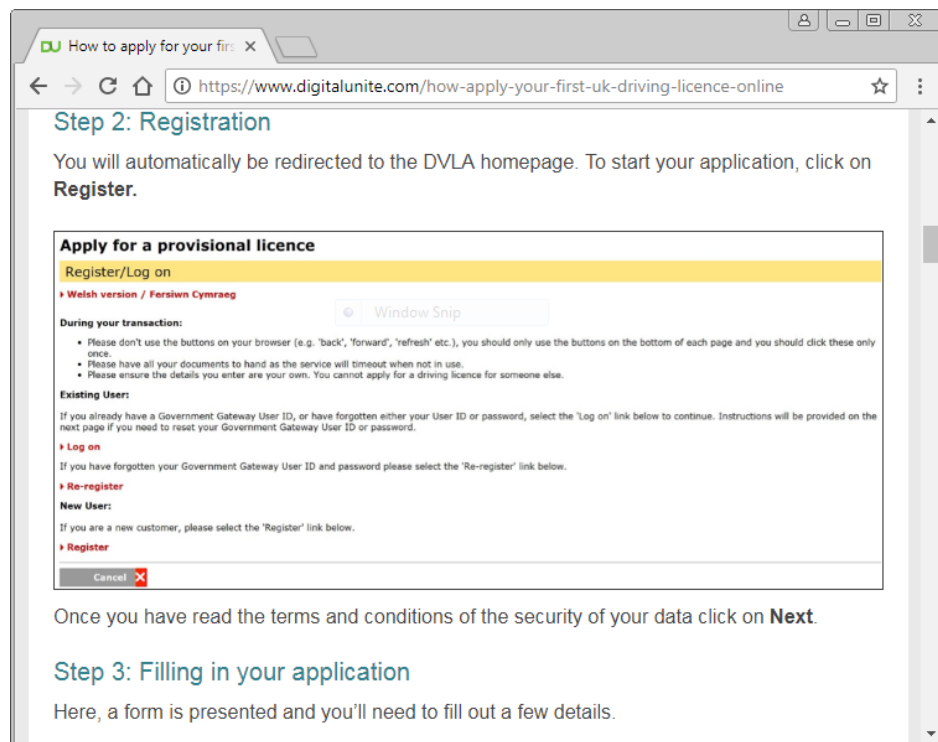
I investigated the training and help that Digital Unite provides in order to teach people how to fill in online forms.

I commented that they only had a small number of resources that they provide to help users fill in forms. My system improves on this as there are an almost endless amount of forms that the user can practise. This is because the admin can create new forms and my system can automatically create new forms from the user's weaknesses.

- **Radio buttons:** These are small circles which you click to select. The one you've clicked will have a dot in it.

☐ Male
☒ Female

I liked this feature of the tutorials because it provided an explanation of how to use each entry type. However, I disliked that there is only a picture of each entry rather than a functioning one that the user can interact with. When I interviewed Steph, she said that explanations on how to use each entry type are not required. Therefore, I have not included any explanation. My system does, however, allow the user to practise any type of question that is in the system rather than just looking at pictures of them.



Digital unite also includes a few full real-life forms for the user to practise. My system includes a large number of forms that the user can attempt therefore it's better than the few forms that Digital Unite offers. However Digital Unite is better than my system in the aspect of form accuracy. The forms that they provide look exactly like real-life forms because they are recreations of them. The forms in my system don't look as similar to real-life forms as the Digital Unite ones do. Improving this is listed in the improvements section.

Official DVSA Theory Test Kit App

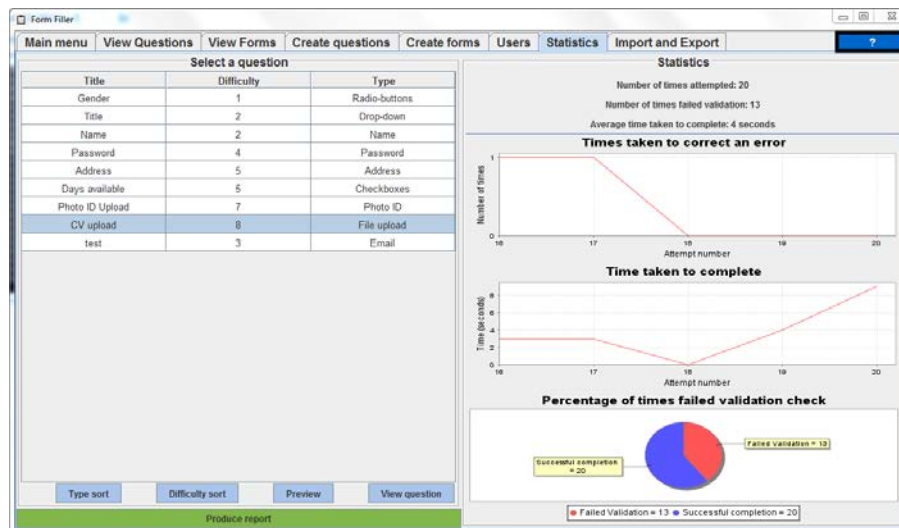


The DVSA Theory Test Kit app provides several progress-tracking features that are similar to what my system provides. I investigated this app as part of my investigation.

I liked the list of topics that the system provides which shows the user what percentage of each topic they have completed and allows the user to select which topics that they want on a practise test. I think that this is a clearer and easier to use implementation than my system because the topics are more clearly grouped and it's an easier way of creating practise tests. However, I think that my system still provides the majority of this functionality because the user can see what percentage of each form they have completed, how many times they have completed it, and the system will automatically create a form that contains the questions that the user struggles with the most with the press of a single button.

The app only shows the statistics as a list like this (pictured to the right). I think that my system improves upon this as it provides a few different views of the same statistics rather than just a list like the theory test kit app.

In my system, the user can view graphs showing a few different statistics for each question in the system. I think that these are easy for the users to use because a lot of them have sight issues so it's easier for them to be able to see the data represented visually rather than having to extract information from a list.



Here you can see some graphs that my system plots to display the statistics for each question. This is easier for the user because, for example, the user can clearly see that there was a large drop in the number of attempts it took them to correct an error on attempt 19. Most users of the system struggle with reading small text. Large diagrams, like these, are much easier for them to interpret.

My system also allows the users to produce a report which summarises the statistics for them and this report can be printed. The theory test kit app doesn't have any such feature. The printable report means that Steph can keep a copy of the statistics that she can view, even when the system is closed or when she's not at her computer. The reports are also dated so Steph can compare new reports with old reports to see how the user's overall progress has changed over time.

bob123's report

Report preview 11.03.2019

Question type	Number of times failed validation	Average number of attempts needed to correct	Average time to complete (seconds)
Address	0	0	6
Email	0	0	0
Drop-down	1	1	15
Radio-buttons	0	0	1
Birthday	0	0	6
Name	0	0	0
File Upload	0	0	0
Photo ID	0	0	4
Calendar	0	0	2
Checkboxes	0	0	4
Password	0	0	4
File upload	1	1	4
Phone			

Print

9.3 Identify Good Features and Shortcomings

Good Features

The different stages of the question builder make it very easy to make a question because they take the user through the process and provide little opportunity for the user to make a mistake.
Form builder is easy to use because all of the buttons are clearly labelled and the screen is well laid out.
A wide variety of forms can be made because of the facility to create questions.
Address auto completion, using the Google Maps API, is realistic to real-life forms and is a feature Steph didn't have before.
File upload is another realistic feature that Steph didn't have before.
Mouse tracking works well to time the amount of time that the user spends on each question because it's done automatically with no user intervention.
The main menu provides quick and easy access to the most recently attempted form.
The details about each question and form are clearly displayed to the user in their respective screens.
The create forms from user weaknesses feature means that the users will always have a new form to practise, even if Steph hasn't created one.
The encryption of the users database means that the sensitive data about each user is safe.
Automatic attendance tracking saves the user's time each session as they don't have to manually mark themselves as present.
The users that have most recently attended being at the top means that Steph can quickly take the register as she doesn't have to search the entire list to find each user.
The statistics screen shows the user's statistics in an easy to understand format using various graphs. This means that the users know exactly how they are progressing at a glance.
The import and export screen allows new questions and forms to be added to the system that were created on different computers. This is essential so that Steph can update the systems.

Shortcomings

Text across the system is quite small. One of the initial requirements was to make the text quite large so that the users with sight impairment could easily use the system. However, I didn't meet this requirement.
Some of the validation, e.g. for photo upload isn't fully working. This was only revealed through extensive testing.
The import and export screen can be a little confusing because the form half of the screen and the question half look very similar so it's hard to tell them apart.
The attempt form window was too long for the low resolution screens of the computer club therefore parts of it was cut off. The screen could be maximised to fix this issue, however.
The "first name" search text field becomes too small to use when the window is resized to a small width. This text field is in the View Users screen.
The view question button doesn't stand out very much in the view statistics screen. This makes it hard to use.
The help button isn't the same size as I wanted from the design. This means that I didn't manage to fully match the designs of the screen from the design.
The system is quite slow to start up. This means that the system seems unresponsive when you first double click the shortcut.
The mouse tracking for the questions is a little unreliable and sometimes doesn't register when the mouse has been put onto a new question.
There are a few inconsistencies with the operation of tables. You can double click to attempt a question in the view questions table but you can't double click in the statistics screen to view the statistics of the question.
The system doesn't recover if the databases are deleted or corrupted. They have to be manually fixed by hand.

9.4 Significant Potential Improvements

Although the system meets all of the requirements, there are a few major ways that it could be improved.

The main issue with the system is that each instance of the system maintains its own database. This means that, when Steph creates new questions or forms, they have to be manually imported one by one on all of the computers in the club. There is also no way of transferring the statistics gathered about each user's attempt of the forms back to Steph's version of the system so that she can analyse it.

Implementing a client-server relationship between the different instances of the system would fix this. There should be a server that hosts all of the databases (questions, forms, forms in progress, question stats, and users). All instances of the system would synchronize with this central database at start-up and would also sync again when they are closed. This means that Steph could create new questions and forms and then push them to the server. When the users start the systems on their laptops, the new questions and forms would automatically be downloaded.

This would also allow me to implement something similar to domain users. The server would act as a domain controller and a user could then log in to any instance of the system with their user account and have their user data fetched from the server. This would mean that the users don't need to make sure that they use the same computer in each session.

The users' statistics will be stored on the server and this means that Steph can easily access all of them and produce reports rather than having to produce the reports from each user's instance of the system.

The next area of improvement is the actual layout of the forms. The forms currently have a layout of 6 questions in a column. This is somewhat realistic to the real world but a greater variety of layouts would make it even more realistic. Many real-world forms have more than 6 questions per page and there are smaller gaps between them. A new form builder that allowed Steph to place questions exactly where she wanted in each form would mean that more accurate and realistic forms could be produced. This could be implemented by allowing Steph to drag and drop the questions exactly where she wanted. Multiple paths through each form could be implemented. For example, if the user answered yes to a particular question, they would be asked a few more related questions however, if they said no, they wouldn't.

The system implements the vast majority of entry types that are in real world forms. However, the main one that is missing is a captcha test. This is a feature of a few online forms. Implementing it would mean that the users would be able to practise performing the captcha test.

It could be useful for the system to have a few built-in option sets that could be selected for a combobox, radio button set, or checkbox set to have. For example, a list of the countries in the world would allow a combobox question to quickly be created that asked the user which country they live in.

The current user report is quite basic and could be improved by showing more statistics. Adding graphs to the report could improve it and make it more useful. The ability for the user to customise what the report shows could also improve the feature. This would mean that the user would be able to create a report to analyse the statistics that they want to see. For example, a report showing the user's progress on completing each question in a particular form or all questions of type gender.

An ambitious feature that could be included is the ability to recreate an online form from a screenshot. The system could be given a screenshot of an online form and it would recreate the form with all of the required entry components with the correct validation. This would make it very quick and easy for Steph to create realistic forms that the users could practise.

9.5 Own Strengths & Weaknesses during Design & Development

Overall, I think that I handled the project quite well and effectively organised myself.

Throughout the project, I worked around 2 weeks ahead of schedule and the deadlines. This meant that I didn't have to rush any aspect of my project and, if any unexpected issues arose, I was able to deal with them in plenty of time. Steph was also very busy and so it took her a week or so to reply to any questions I asked her about the system, e.g. during testing, and so I was able to provide her with plenty of time to reply because I didn't have the deadline very close. This all resulted in an overall better system and documentation. When working on future projects, I will aim to work ahead of schedule again because it worked so well for this project.

I also had a strong work ethic during the project. I made sure to complete each section to the best of my ability and made sure that I had enough time to do this. This meant that I was able to implement all of the features of my system that I intended to. The only feature that I excluded was the ability for the system to automatically determine the reason for a user missing a session. This is because the feature was not necessary anymore and wouldn't add much to the system. For each feature that I did implement, I spent the time to code it properly so that it wasn't just 'hacked' together and so had fewer bugs.

The scope of my system was too large. I wanted to create a system that would fix all of the issues that Steph had with the current system and would improve on quite a lot of them significantly. Although I managed to finish it, I was under an immense workload the duration of the project. Very often, I had to work late into the night to get features finished so that I would remain on schedule. My write-up documents were also very large as I had to document the entirety of my large system. This was probably my biggest weakness during the coursework and is something that I will try to avoid in the future.

My system was also bespoke and there were quite a few components and features that I had to code entirely from scratch, without being able to leverage the standard libraries. For example, there was no built-in facility in Java to save GUI components to file. This meant that I had to create new subclasses of all of the GUI components that I wanted in forms so that I could save the forms to file along with the data entered into each one. This took a large amount of time and testing. A smaller project scope would have meant that I could more fully develop these features and potentially add even more new functionality to the GUI components.

I extensively tested my system during development and after finishing development. This meant that the system had very few bugs. This improves the usability of the system because there were fewer annoying and unpredictable issues. This was important because the members of the club are not highly skilled in using computers and so would be very confused by, and probably would not be able to work around, bugs. I think that spending the time to look over the system and test it was time well spent and in future projects I will do the same. This is to ensure that what I produce is high quality and well-polished.

I had quite a lot of resilience during development. When I found a bug, I managed to fix it in all instances. This is because I worked on fixing it until it was fixed, even if it seemed impossible and took a while to fix. An example of this was making the text wrap automatically in the tables. This took around two days of work to do because, surprisingly, it wasn't a built-in feature of tables, even though I assume it's a common use case. The text wrapping meant that I was able to accommodate for all lengths of titles and descriptions in the questions and forms tables. This resilience meant that my system met all objectives because I didn't have to remove any on the grounds of them being infeasible. I think that this resilience is a skill that I can carry over to work that I do in other areas and subjects.

Despite how useful having resilience was, sometimes I didn't take a break when I should have. For a few problems, I worked on them until they were fixed in one continuous session rather than taking breaks. In all these cases I actually only managed to fix the problem once I had taken a ten minute break. If I had taken the breaks earlier then I would have solved most of these issues more quickly. In future, I will take more frequent breaks, especially when trying to solve a problem, because it will save me more time overall.

When I was doing the written documents for my system, I often wrote too much and wrote with quite a lot of repetition. This meant that it took me a long time to complete the documents, especially the design document. Writing more effectively and efficiently would mean that I would have been able to complete the written documents in a much smaller amount of time. It would have also meant that the documents were higher quality and easier to read.

I had a very systematic approach to working on the project. For example, I created a standardised form for my command line output. This was "[INFO/ERROR] <CLASS_NAME> message". I added these to each method. This meant that, when there was a bug, I could easily identify where the error occurred and what methods ran before and after it. This saved a lot of time during development because I didn't have to spend time hunting for where each bug occurred. I will adopt a similar systematic approach during projects in the future because it worked so well during this project.

I managed the project using the online task management system "Trello". This meant that I always knew exactly how much of the project I had completed and what I needed to do next. I had a card for each objective and, attached to each card, was a checklist of what I needed to do to finish work on that objective. I also coloured each card based on whether: I had started working on it, had finished working on it, or was currently working on it. Without Trello, I wouldn't have been able to complete the project as well because it would have been difficult to manage my 60+ objectives. Using Trello, I was able to see my progress through the entire project at a glance. In future, I will continue to manage my time using Trello because it allows me to make efficient and effective use of my time.

9.6 Future Changes of Approach to Avoid Problems

There are a few changes to my approach that I would make in the future.

The first of which is to spend the time evaluating each of my objectives during the design phase to determine whether they are really necessary. My system has a few features that will be seldom used by the computer club and Steph. This is because I didn't ensure that they were 100% necessary before implementing them.

The main features that are unnecessary are most of the user management features. There really only needs to be a facility for the system to differentiate between different users so that their progress through the forms and questions can be stored separately. Storing large amounts of personal information about the users (date of birth, phone number, etc.) isn't needed. Attendance tracking is also not needed.

Developing these features took away time from developing and testing other features and also meant that there were more areas that could introduce bugs and issues into the system.

One of the main issues that I had to solve in this area was storing the user information into three separate databases, two unencrypted and one encrypted so that, if one was corrupted, the others weren't affected. The difficulty was managing the user list class and structuring it so that users could be loaded from file and reassembled correctly from the three different databases. The complexity of this area of the system caused a few difficult-to-solve bugs. For example, the system would load a user from file from the username file but wouldn't load the question stat database associated with the user and so, when the user attempted a question, the system wouldn't work. It was hard to identify that the user's question stat database hadn't been loaded because, before I implemented the three database system, there was no way that the user could be loaded without loading their question stat database.

I spend a lot of time during development ensuring that the system was fully resizable and adapted to different screen sizes and resolutions because I knew that the users at the club all had different laptops. I tested the system on the computers in college which had 768p monitors, on a 720p monitor, on a 1080p monitor, and on a small laptop. Despite this, the system still didn't scale properly on some of the laptops in the club. In future, I should test on some even smaller screens (sub-720p) to be absolutely certain that my system would work on all computers.

I also didn't test the system on some very slow computers as all of the computers that I had access to were relatively modern and fast computers. This meant that some aspects of the system performed slower than I would like on the slow laptops that the computer club members had, most notably, the time taken for the system to start up. In future, when developing for slow computers, it would probably be best to actually get hold of a computer/laptop similar to the ones that the users in the club would be using so that I can ensure that it runs well and as I would like it.

When producing write-up documents in the future, I will avoid repetition to save time. I've mentioned the benefits of this above.

I didn't spend the time to properly structure all of my text files. I came up with the format for them myself rather than using an already established format such as JSON or XML. This meant that I had quite a lot of issues to do with writing and loading data to and from text files.

There was a large amount of different fields of data that I had to store in the forms in progress files. This was difficult to structure as a lot of the data was nested inside other data. For example, each form would have a list of questions and then each question in that list would have its own set of data such as the data that the user had entered into the question. This meant that I had to use a large number of

different delimiters. Near the end of the project I started to run out of delimiters and so had to use characters such as "&". This meant that the text files were very complex and easy to corrupt. These text files also stored what the users had entered into the questions therefore, if the user had entered any characters that also happened to be a delimiter, the system would be unable to load the text file and so would crash and all data would be lost.

I managed to solve this problem by making the system escape all characters that the user entered that were also delimiters. Rather than just removing the character the system replaces it with a "safe" character before saving to file. When the data is loaded from file, the system replaces these "safe" characters with the original characters again. To the user, this process is invisible. This means that users can enter any character that they want without corrupting the database.

In future, however, it would have been much faster to just use a standard file structure system such as JSON as it already solves all of these problems and there are many well-tested and prewritten libraries that can be used.